Campus Heilbronn
TUM School of Computation, Information and Technology
Technical University of Munich
Prof: Tobias Ribizel
E-mail: tobias.ribizel@tum.de
Introduction to C++ Programming

In-Memory SQL Database

Introduction

This project implements a small, dependency-free in-memory SQL engine in C++23. The design separates the library core from the front ends (CLI and Qt GUI). The front ends only handle input and output, while the core contains all database logic. This ensures the system is testable, UI-independent, and reusable.

Library Core and Data Model

The core data model is centered around a Database object managing multiple tables. Each table consists of a schema, a name-to-index map for constant-time lookup, and a vector of rows. Rows are defined as std::vector<Value>, where Value = std::variant<Int, Str>. This approach enforces compile-time type safety without virtual dispatch and stores rows contiguously for efficient scans. In summary, the model is simple but guarantees type safety and predictable performance.

Parsing Pipeline:

The parser is a hand-written recursive-descent parser. Input SQL is split into statements, tokenized, and converted into AST nodes such as Select, Insert, CreateTable, Update, and Delete. Each node has explicit fields, for example, a Select node includes projection, table name, and optional WHERE or LIMIT clauses. Errors are reported with exact source positions, making diagnostics clear. Thus, parsing remains lightweight, dependency-free, and precise.

Execution:

Execution is performed through overloaded Database::exec methods, one per AST type. Insert checks schema compatibility before appending a row. Select scans tables, applies filters with std::visit, and builds result sets. Update and Delete also scan rows, applying conditions to modify or remove data. This design ensures every statement type has a dedicated, type-safe execution path.

Flow Of Code:

SQL input → Parser → AST → Executor → Database → Result.

Testing:

The project includes unit tests to verify correctness and robustness. Using Catch2 and GTest, test cases were written for creating tables, inserting rows, selecting with and without predicates, updating and deleting data, and handling invalid queries. Negative tests confirm that errors are reported for schema mismatches, unknown identifiers, or malformed SQL. Compilation with(-Wall -Wextra) and sanitizers (-fsanitize=address,undefined) ensures that the implementation is safe, correct, and free from memory errors.

Error Handling:

The engine ensures that **invalid inputs never crash the system**. During parsing, errors are reported with descriptive messages and exact source positions, allowing the user to quickly identify mistakes. In execution, schema checks prevent type mismatches, such as inserting a string into an integer column. If a query references an unknown table or column, the executor returns a structured error instead of continuing. This design guarantees that incorrect queries fail safely and predictably.

C++ Features and Conclusion:

The implementation leverages modern C++: std::variant and std::visit for values, std::optional for clauses, std::string_view for efficient tokenization, RAII for safe memory handling, and move semantics to avoid unnecessary copies. Overall, this project demonstrates a compact, type-safe, and testable in-memory SQL engine built entirely with modern C++ design principles